Computer Organization

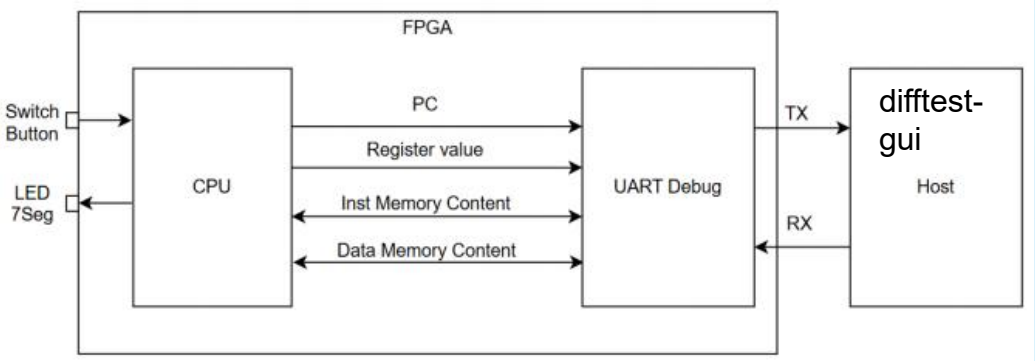
Lab9:CPU introduction

!! This document is for internal use only at Southern University of Science and Technology. Please do not repost it on other public platforms without permission from the instructor!!

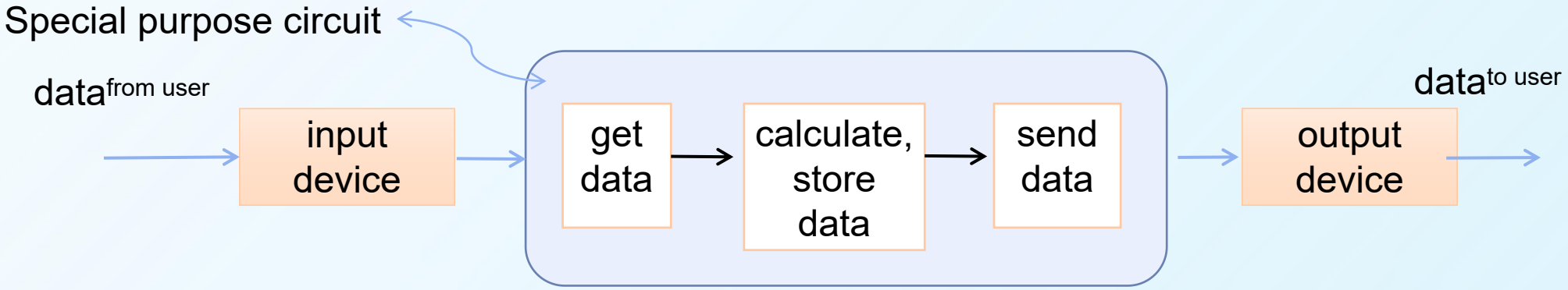
CPU tests

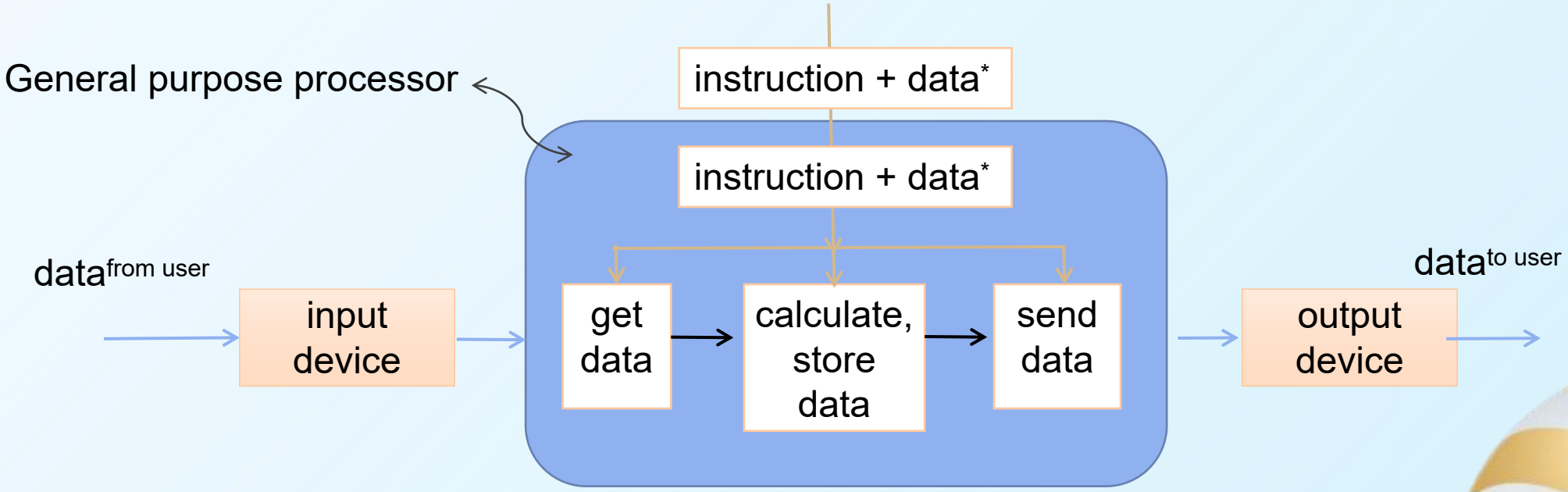
Topic

- **Special purpose circuit** vs **General purpose processor**
- **CPU tests**
  - **CPU with I/O on the development board**
  - **CPU with Difftest-Gui**
- **Practice**
  - P1: **Test CPU using the IO devices of the development board(EGO1)**
    - **RISC-V**(MMIO), **a tailed CPU**(MMIO)
  - P2: **Test CPU using "difftest-gui" work on Host**
    - **RISC-V**, **a tailed CPU**, and **difftest-gui**

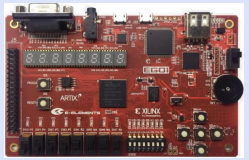
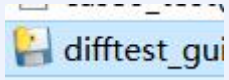
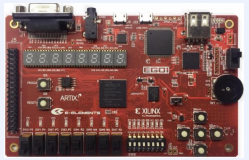


Special purpose circuit vs 'General purpose processor'





Experimental Task and Toolkits

Task	Tool kits
Design and implement a digital circuit on FPGA (CPU is also a digital circuit)	➢ **Vivado** ➢ **FPGA based Development Board EGO1** 
Test the **CPU** with **program(s)** (Both CPU and the program(s) are based on **RISC-V/LoongArch)**	➢ **Assembler (Rars/ Lars)** / ➢ **Difftest_Gui**  ➢ **Vivado** ➢ **FPGA based Development Board(EGO1)** 

Demo1 : Lighting led(s) by 'Special purpose circuit'

Design file(in verilog) + Constraint file

```
//Design file by verilog, save as sw2led.v
module sw2led(sw,led);
        input    [ TBD :0] sw;
        output [ TBD  :0] led;

        assign led = sw;
endmodule
```

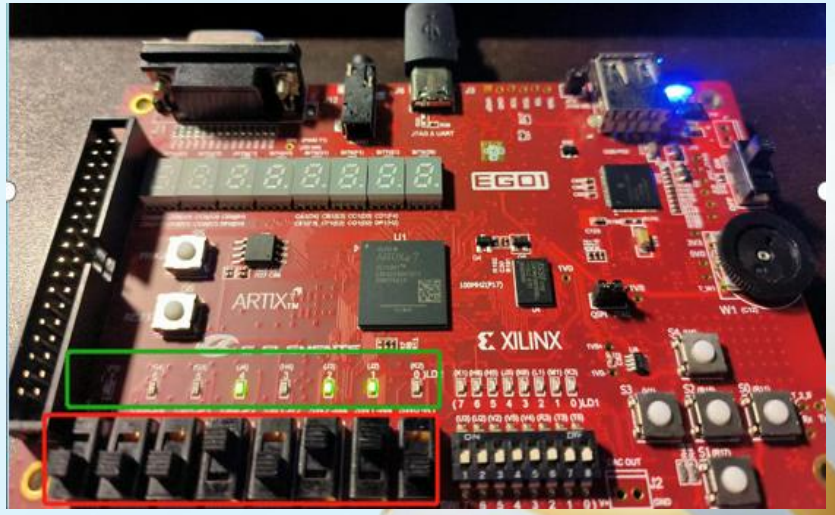
```
#Constraint file, save as sw2led.xdc
set_property IOSTANDARD LVCMOS33 [get_ports {led[ ]}]
...
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[ ]}]
...
set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN F6 [get_ports {led[ ]}]
...
set_property PACKAGE_PIN K2 [get_ports {led[0]}]
set_property PACKAGE_PIN P5 [get_ports {sw[ ]}]
...
set_property PACKAGE_PIN R1 [get_ports {sw[0]}]
```

----(Vivado generate bitstream)---> **bitstream** file

---(Vivado hw manager, open Target, program device)

---> Implement the circuit on the **FPGA chip**

---> **Test** the circuit on the **FPGA based Development Board**



Demo2: Lighting led(s) by ‘General purpose processor’

1. **The program (instructions and data*)** : file in hexdecimal named ‘sw_led_test(onlyX1writtable_X31value_FFFF0000).txt’

   - The file in “labs/lab9” directory of course blackboard site.

2. **The ‘General purpose processor’ :** a bitstream file named ‘TailedCPU_onlyX1writtable.bit’(a Tailed General purpose processor)

   - The file in “labs/lab9” directory of course blackboard site.

3. **Testing steps**:

1) Let the given ‘tailored General purpose processor’ work on the FPGA chip.

   ✓ use “program device” feature of Vivado

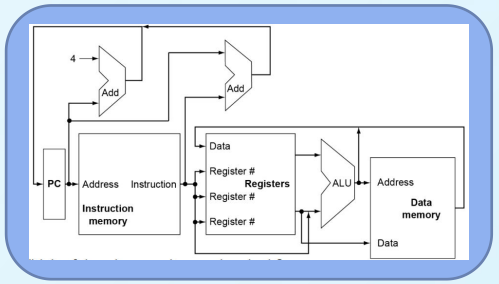
2) Load the program to the ‘tailored General purpose processor’.

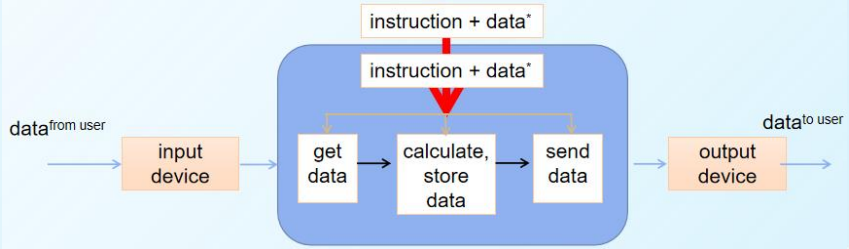
   ✓ use “Load instruction hex” feature of “difftest_gui”

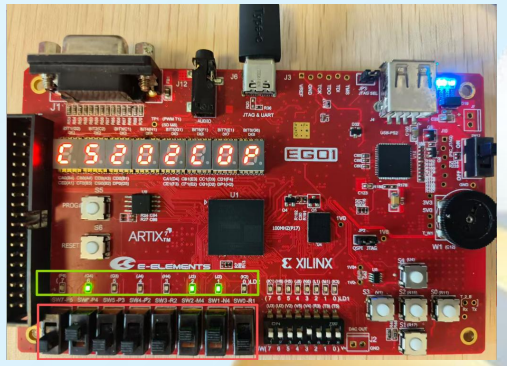
3) Test the ‘tailored General purpose processor’ loaded with a program.

   ✓ Switches and Leds on EGO1

instruction + data*





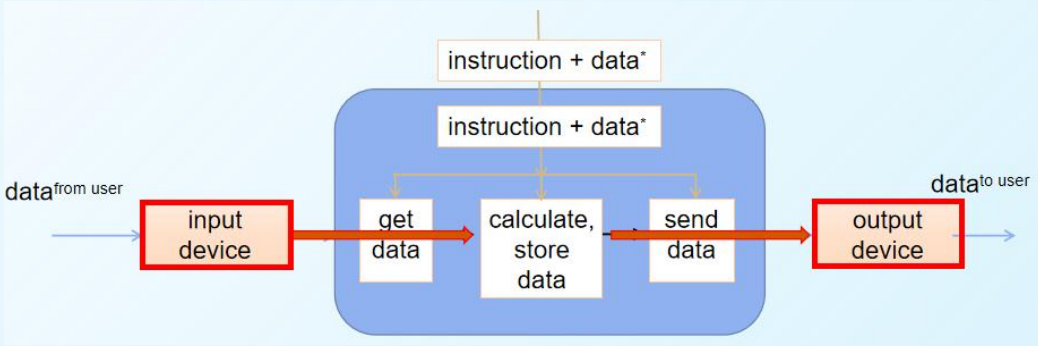


Demo2-1: The 'tailored General purpose processor' with MMIO

- ✓ **Based on RISC-V32I**

- ✓ **Data flow**
  - ✓ the data **MUST be stored into the register(s)** of 'tailored General purpose processor' before be calculated or be sent to the output device.

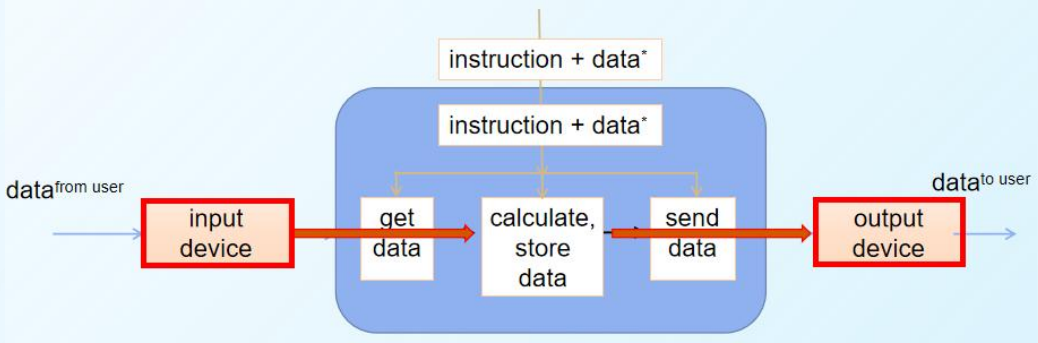
- ✓ **Register(s)**
  - ✓ There are total 32 registers, the width of each register is 32bits.
  - ✓ **ONLY x1** is writable.
  - ✓ The value in **x31** is **0xFFFF_0000**, the initial value of other registers is **0**.

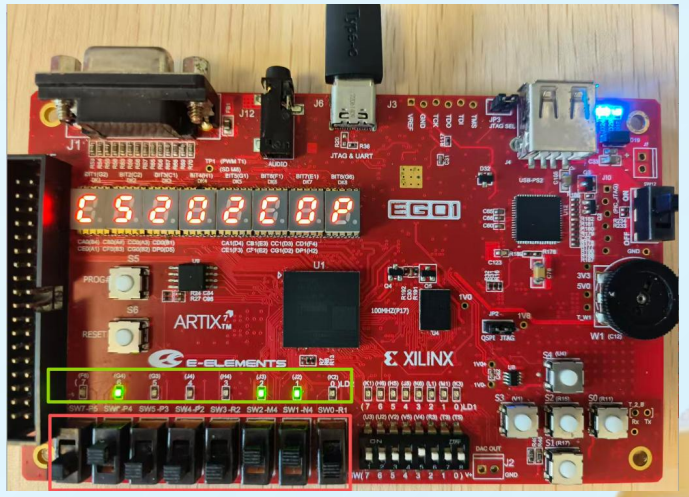


- ✓ **I/O and address(MMIO: Memory Mapped IO)**
  - ✓ using "load" instruction to get data from input device(addressed by 0x**FFFF0000**) **to Register**
    - ✓ **read from 0XFFFF0000 is to get the data from 8 switches**
  - ✓ using "store" instruction to send data from register to output device(addressed by 0x**FFFF0008**)
    - ✓ **write to** 0x**FFFF0008 is to send the data to 8 leds**

Demo2-2: The program(instructions and data*) with MMIO

```
#assmbly source file  in RISC-V
.data                 # data* is 4bytes, its initial value is 0
        buf: .word 0x0000

.text                 # instructions
main:
        # in  'tailored General purpose processor'
        # the value in register x31 is 0xFFFF0000.  ONLY register x1 is writable

        lw   x1, 0(x31)      #copy data from 0xFFFF0000(switches) to register x1
        sw   x1, 8(x31)      #copy data from register x1 to 0xFFFF0008(leds)

        j main               # jump to the instructions labled by main
```

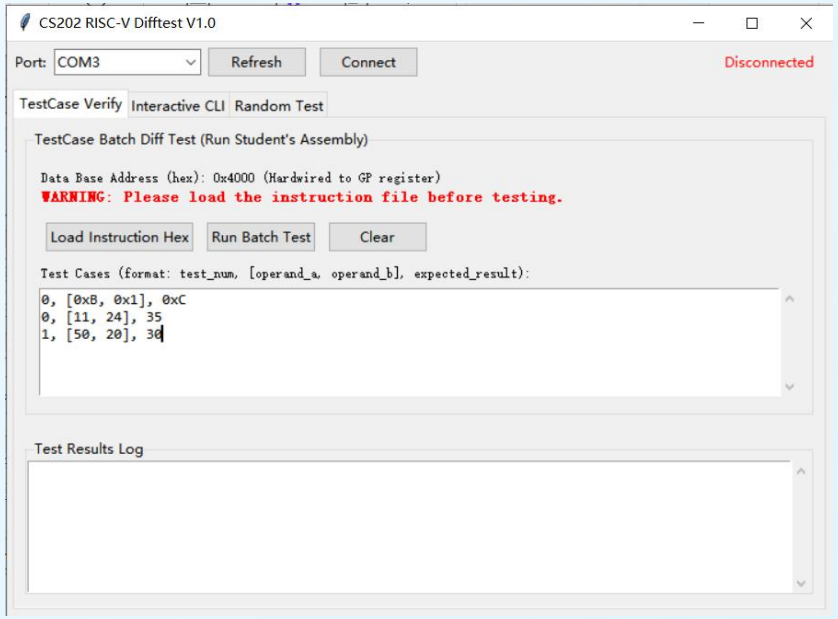




Demo2-3: Load the program to the ‘tailored General purpose processor’

Two methods:

- ✓ 1. Load the program into the CPU when generating the bitstream (would be introduced on lab10)
- ✓ 2. Load the program into the CPU through the UART interface.
  - ✓ difftest-gui: It is a UART-based differential testing tool, featuring **UART communication**, loading hexadecimal instruction machine code, case verification, and other functions.



Tools: difftest-gui(1): Uart connect

Preparation:

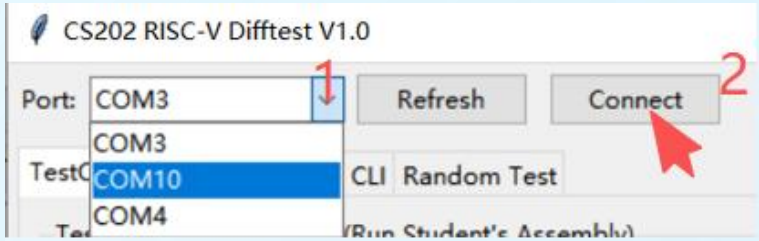
1. The development board(EGO1) is connected to the USB port of the computer via UART/JTAG interface, and the development board is powered on.
2. Load the bitstream file of the circuit onto the development board's FPGA (via "program device" in Vivado), and this circuit possesses UART communication functionality.
3. Find the port name by "Device Manager" of OS.
4. Run difftest

Uart Connect:

1. Choose port name

tips: if the port is not appear, click on "Refresh"

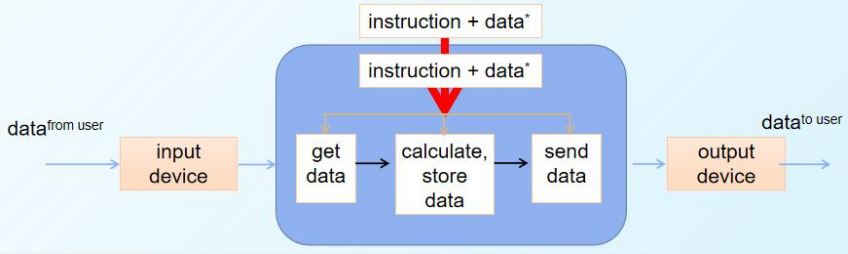
2. Click on "Connect" .



Tools: difftest-gui(2):Load instruction hex

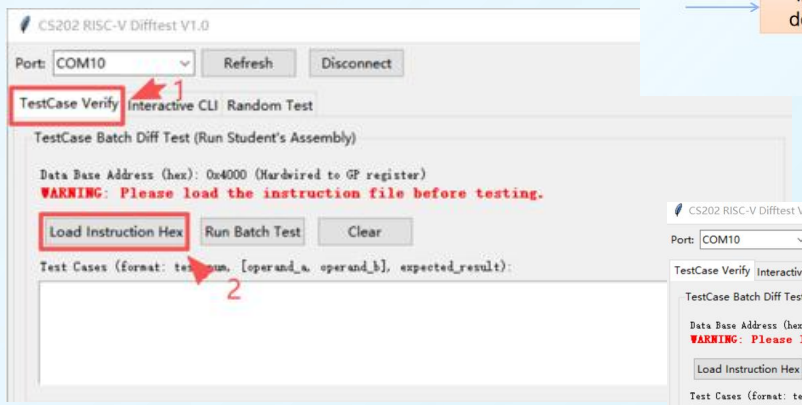
Preparation:

- A CPU supporting UART communication has been loaded into the FPGA chip of the development board.
- an instruction hex file.
- UART port connected (steps on previous page)



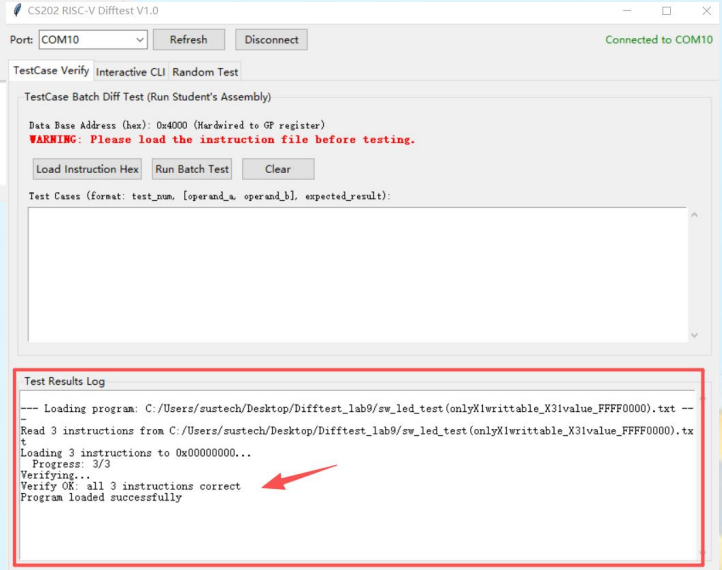
Load instruction hex file to CPU:

1. Click on “Testcase Verify” page
2. Click on “Load Instruction Hex”



Check whether the instruction has been successfully loaded

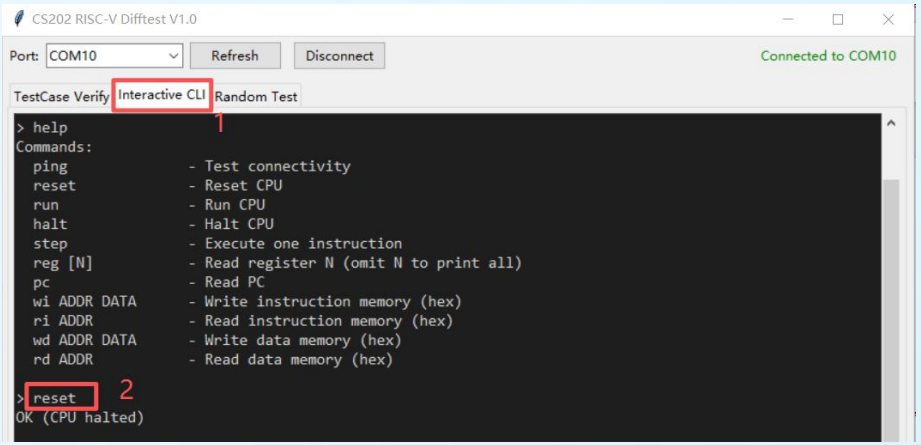
```
#assmbly source file  in RISC-V
.data              # data* is 4bytes, its initial value is 0
        buf: .word 0x0000

.text              # instructions
main:
        # in  'tailored General purpose processor'
        # the value in register x31 is 0xFFFF0000.  ONLY register x1 is writable

        lw   x1, 0(x31)      #copy data from 0xFFFF0000(switches) to register x1
        sw   x1, 8(x31)      #copy data from register x1 to 0xFFFF0008(leds)

        j main               # jump to the instructions labled by main
```



Tools: difftest-gui(3):reset & run

Preparation:

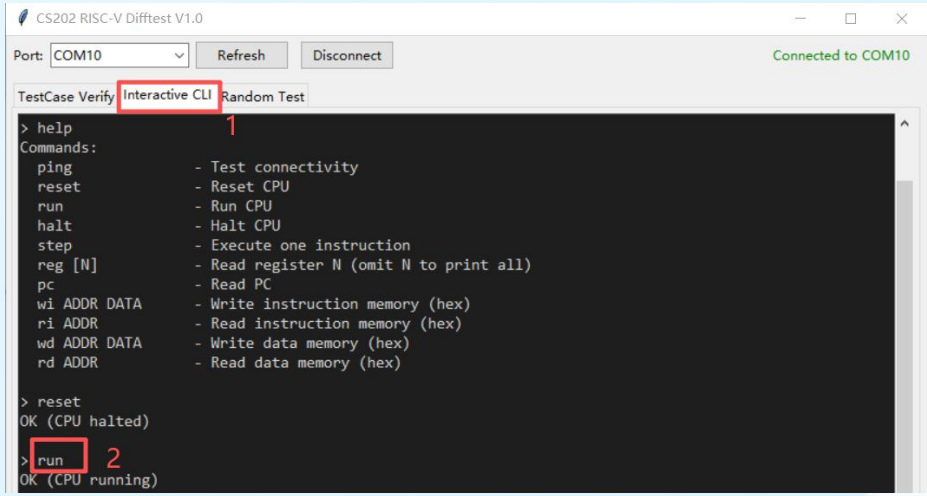
- The CPU supporting reset and run.
- The new program has been load to the CPU.



Reset:

- Reset the internal state of the CPU. Note: In this demo, the value(0xFFFF0000) is written to x31 by hardware encoding, it will not be affected by resetting.

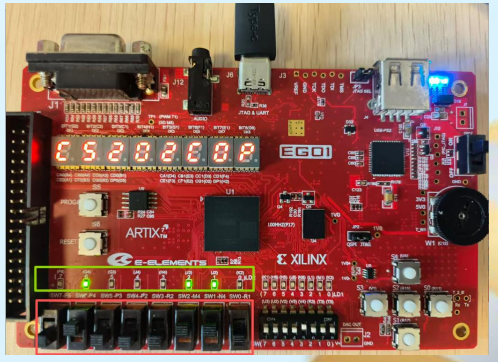
1. Click on **"Interactive CLI"** page
2. Enter the command "**reset**" and press Enter to execute



Run:

- Start the CPU and begin executing according to the newly loaded instructions.

1. Click on "**Interactive CLI**" page
2. Enter the command "**run**" and press Enter to execute



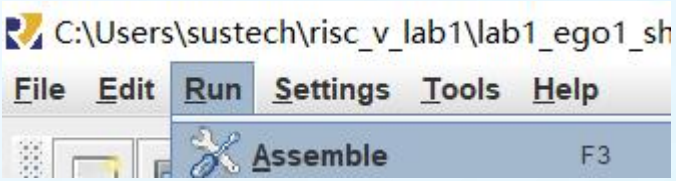
Tips: generate the instruction hex file

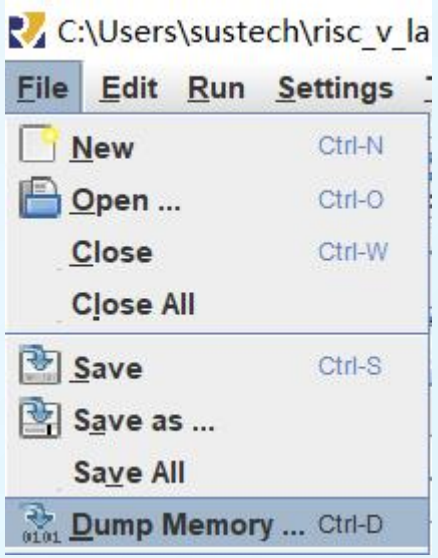
Generate the machine code file(s) in **Rars**

1-1. Open the asm file

1-2. Assemble the asm file to the machine code

1-3. Dump machine code and data in avaliable format.





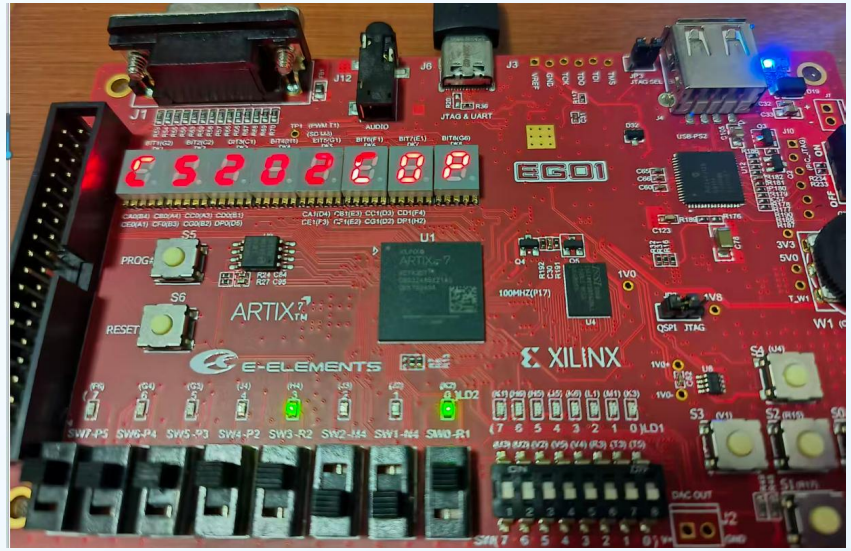
1-3-1. Dump **instructions** in **Hexadecimal Text**

NOTE:

1. The instruction dump file is suggested to be named as “xxxx.**txt**”

2. The operation steps on “Lars” are consistent with those on “Rars”.

Practice1(1) ‘special purpose circuit’ vs ‘General purpose processor’



Process the 8-bit input from the switch, set the upper 4 bits of the data to 0, retain the lower 4 bits, and display the processed result on the LED lights.

e.g. input is 8’b1111_1001， output is 8’b0000_1001.

- Assembler the new asm file with ‘Rars’, then dump the instruction hex file.
- Using “difftest-gui” to load instruction hex which is generated by previous step, then invoke reset and run.
- Do the test on EGO1.

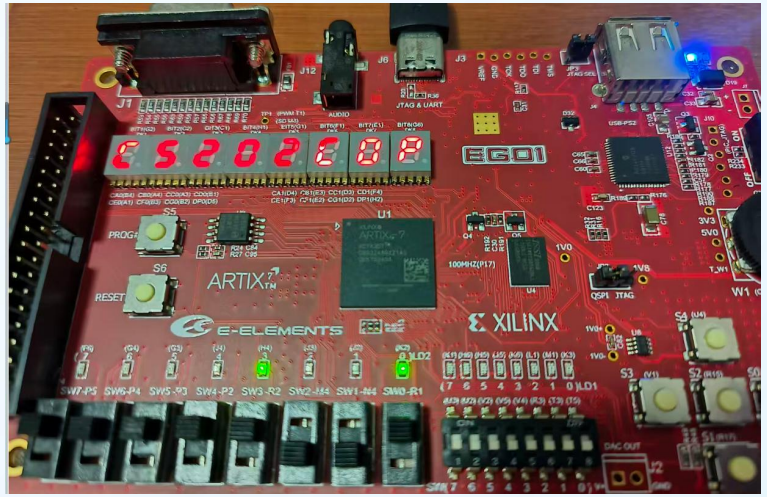
```
.data                    #A

.text
start:
        lw   x1,0(x31)
        addi x1, x1,0x0F
        sw  x1,8(x31)

        j start
```

```
.data                    #B

.text
start:
        lw   x1,0(x31)
        ori x1,x1,0x0F
        sw   x31,8(x31)
        j start
```

```
.data                    #C

.text
start:
        lw   x1, 0(x0)
        slli  x1,x1,4
        sw  x1, 8(x31)

        j start
```

```
.data                    #D

.text
start:
        lw   x1,0(x31)
        andi x1,x1,0x0F
        sw  x1,8(x31)

        j start
```

Practice1(2) ‘special purpose circuit’ vs ‘General purpose processor’



- To implement the same logical relationship between inputs and outputs in this test(Process the 8-bit input from the switch, set the upper 4 bits of the data to 0, retain the lower 4 bits, and display the processed result on the LED lights), which of the following processes are needed on Demo 1(Lighting the led by ‘Special purpose circuit’)?

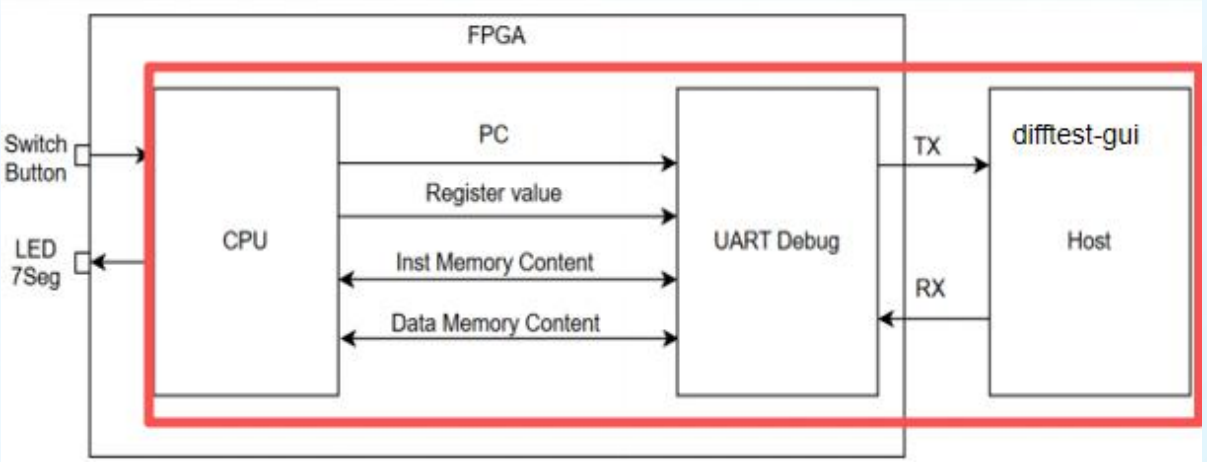
1) update the design source file
2) update the constraint source file
3) update the testbench file
4) regenerate the bitstream file
5) re-program the device with the new bitstream file
6) using a new FPGA chip to be programmed
7) reset the chip type in the vivado project

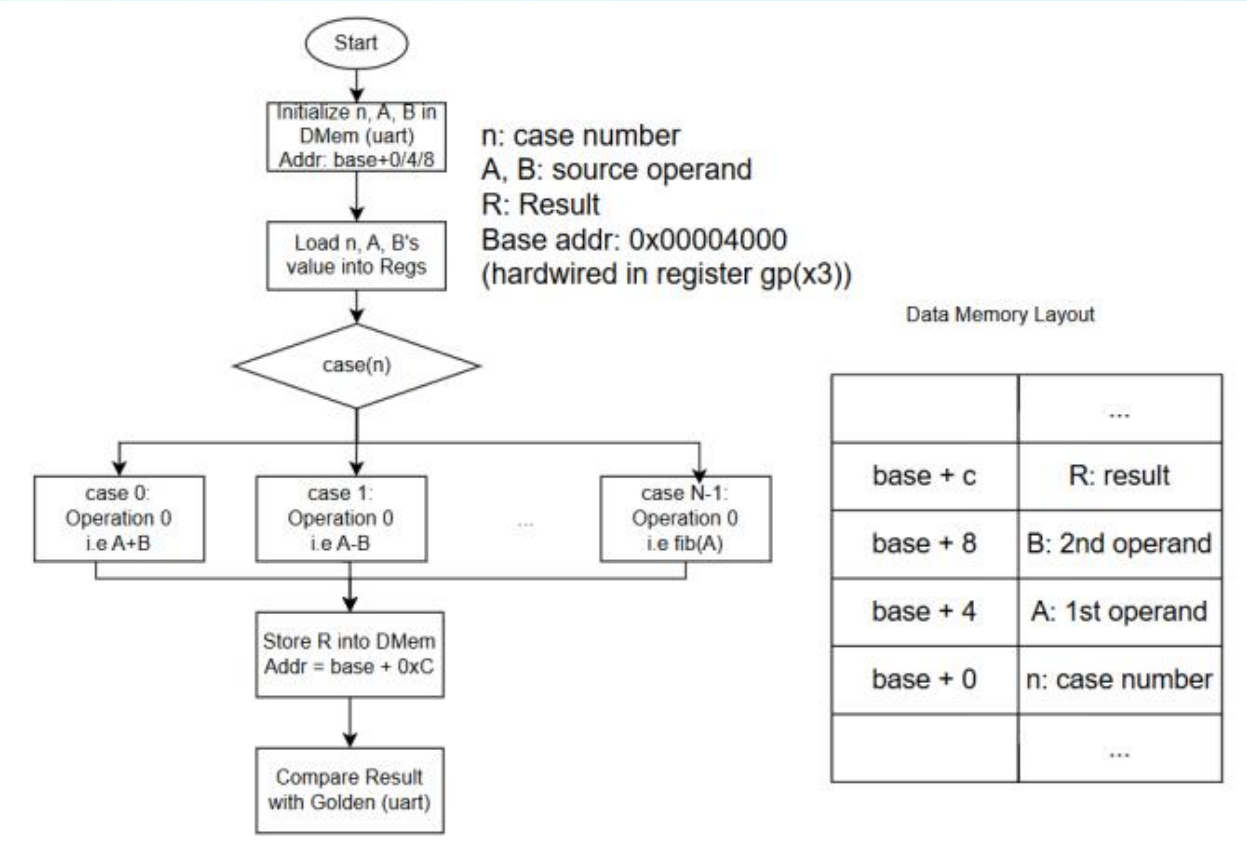
A. 1,2,3,4,5,6,7 **B**. 6 **C**. 7 **D**. 1,2,3,4,5 **E**. 1,2,4,5 **F**. 3,4,5 **G**. 1,4,5 **H**. 2,4,5 **I**. 3,4,5 **J**. 1,4,5,6,7

CPU test with difftest

Features:

- ✓ The CPU needs to interact with the "UART Debug".
- ✓ Does not involve conventional IO devices processing
- ✓ Achieve batch automatic comparison testing





Demo3: Use “difftest” to perform batch automatic testing of CPU

1. **The program (instructions and data*)** : file in hexdecimal named ‘case0_test(onlyX1writtable_casebase_4000).txt’

   - The file in “labs/lab9” directory of course blackboard site.

2. **The ‘General purpose processor’ :** a bitstream file named ‘TailedCPU_onlyX1writtable.bit’(a Tailed General purpose processor)

   - The file in “labs/lab9” directory of course blackboard site.

3. **Testing steps**:

1) Let the given ‘tailored General purpose processor’ work on the FPGA chip.

   ✓ use “program device” feature of Vivado

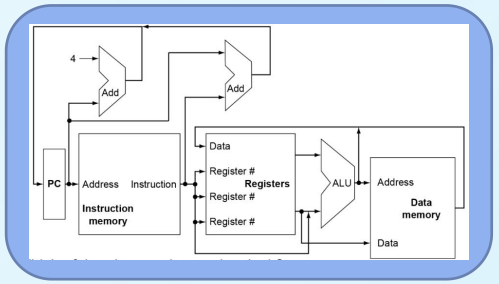
2) Load the program to the ‘tailored General purpose processor’.

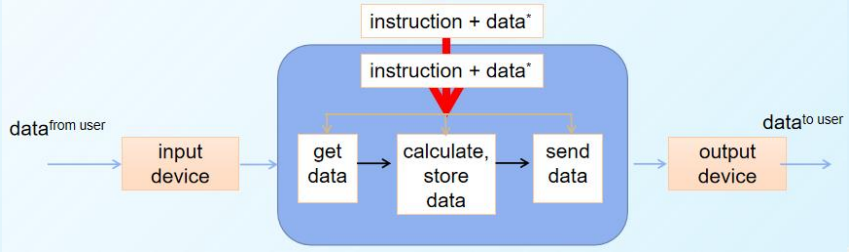
   ✓ use “Load instruction hex” feature of “difftest_gui”

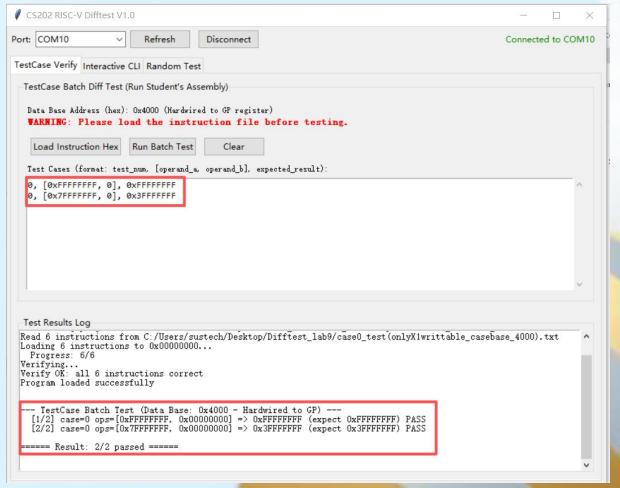
3) Test the ‘tailored General purpose processor’ loaded with a program.

   ✓ use “Testcase Verify” of “difftest” to perform batch automatic testing.

instruction + data*







Demo3-1. test dataset in “difftest”

1.**Testcase in the program**(‘case0_test(onlyX1writtable_casebase_4000).txt’

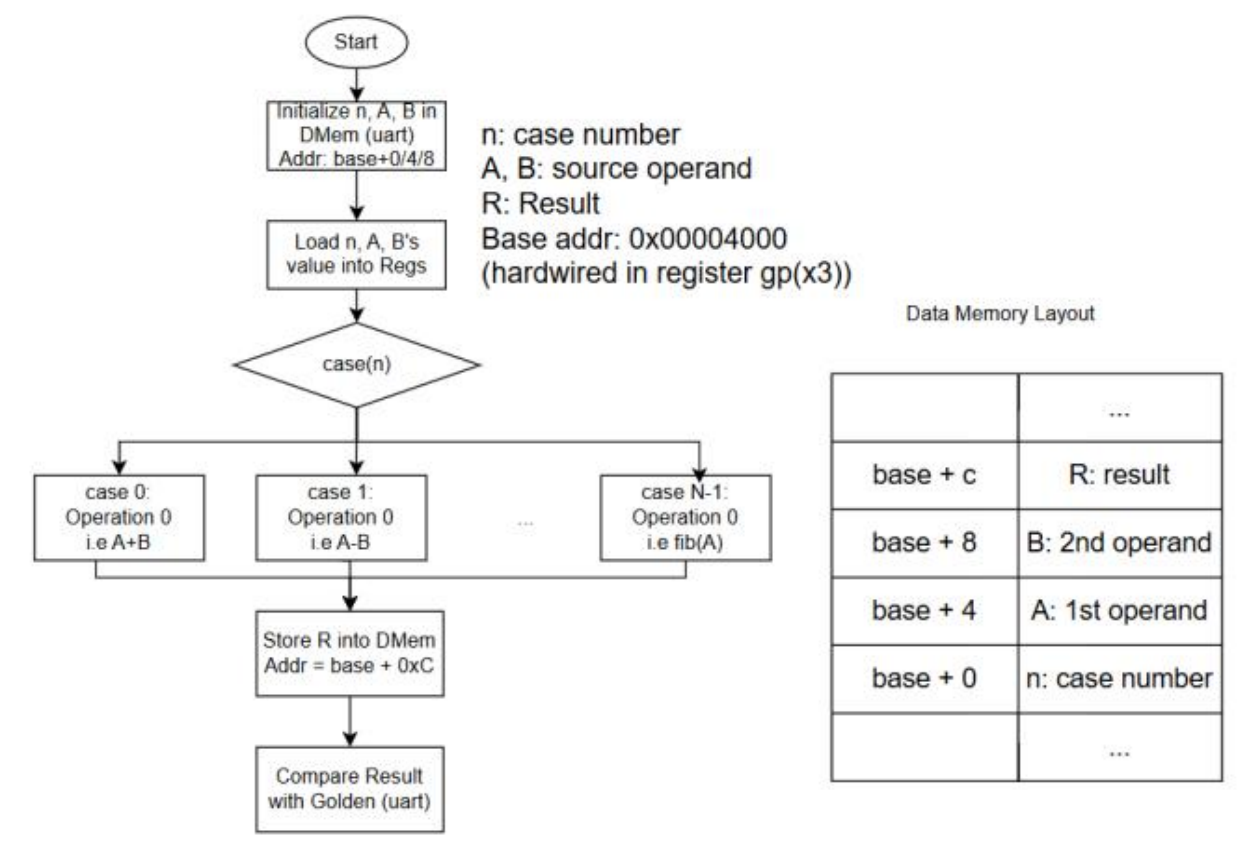
- ✓ Only 1 testcase, numbered 0
- ✓ The target instruction for the test is:
  - ✓ srai x1, x1, 1

2. Test dataset based on Testcase

- ✓ test dataset format

```
Test Cases (format: test_num, [operand_a, operand_b], expected_result):
```

- ✓ a demo for test dataset:
  - ✓ 0, [0xFFFFFFFF, 0], 0xFFFFFFFF
  - ✓ tips:
    - ✓ **Use half-width English characters**
    - ✓ For I-type instruction, operand_b here is 0



Demo3-2. input test dataset & “Run batch Test” in “difftest”

Prepare

1. The FPGA chip of the development board that has been program deviced with CPU’s bitstream.

2. The “diffset” has been indirectly connected to the CPU through the UART port.

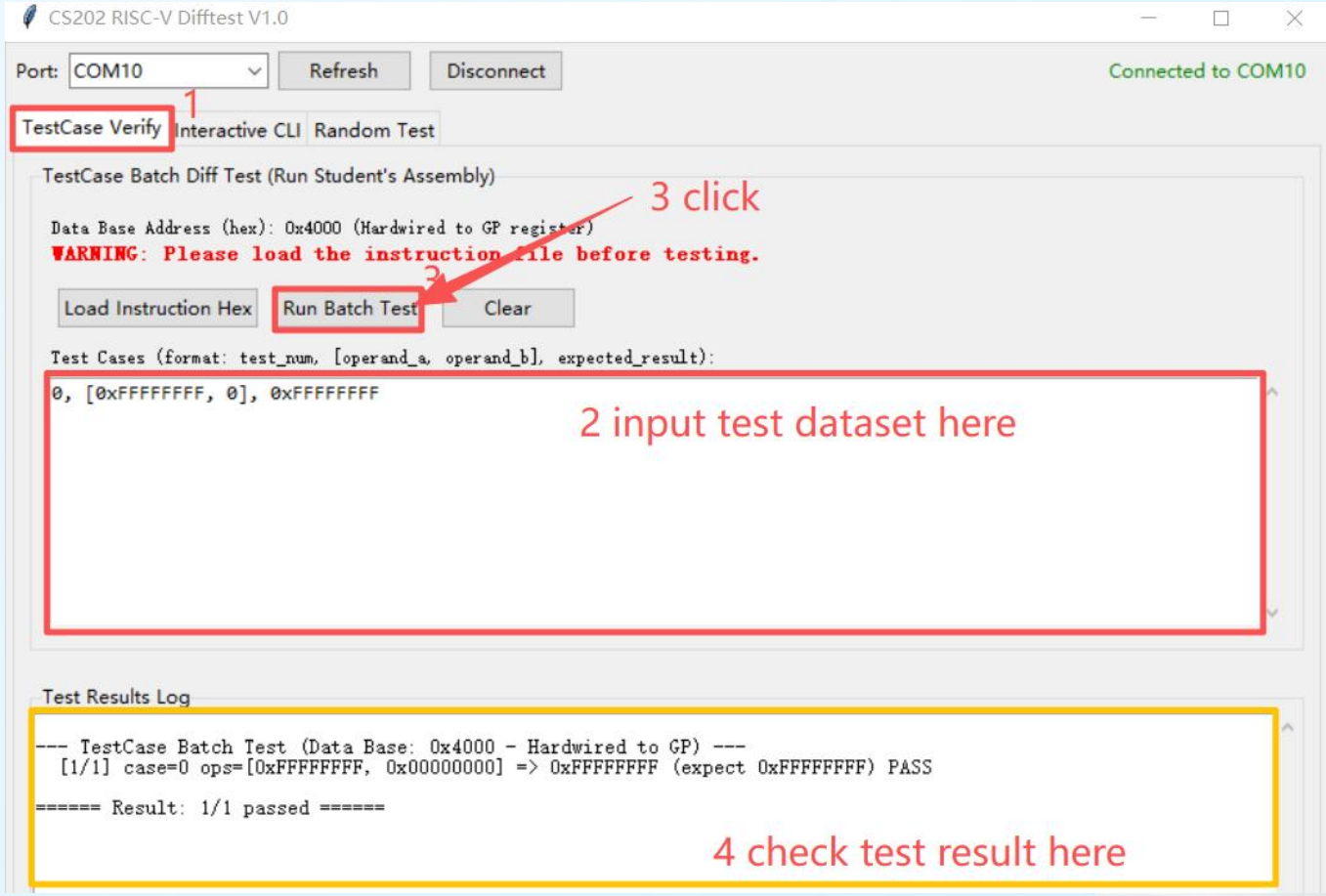
3. Load the program(“Load Instruction Hex”).

Steps

- ✓ 1. choose “TestCase Verify” page
- ✓ 2. input test dataset

  Each test data item on a separate line

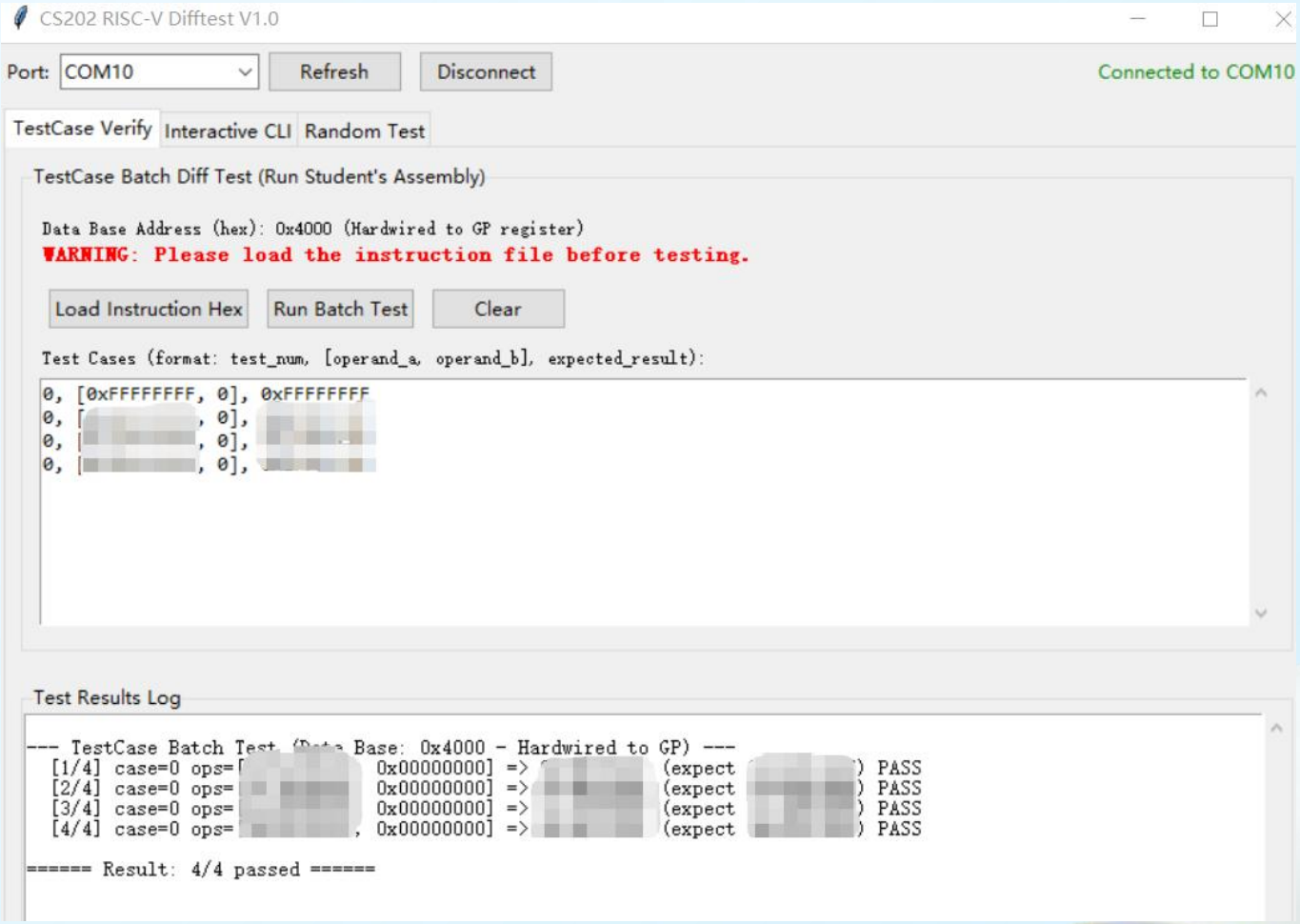
- ✓ 3. Run Batch Test
- ✓ 4. Check test result



Practice 2. Complete test dataset & “Run batch Test” in “difftest”

Refer to the method on the previous page,

1. complete the test dataset based on the target instructions(“srai x1, x1, 1”) , the test dataset should includes at least 4 different test data item.

2. use the “difftest” for batch testing

3. verify the functionality of the target instruction.(“srai x1, x1, 1”)